

Documentation: Implementation of Dijkstra's Algorithm for Indian Cities

1. Title

Implementation of Dijkstra's Algorithm for Finding the Shortest Road Distance Between Indian Cities

2. Introduction

In many real-world applications, we need to find the shortest or least-cost route between different locations. For example, when travelling from one Indian city to another, there may be several possible routes. Each route can have a different road distance.

To solve this type of problem, we can represent cities and roads as a **weighted graph**.

In this project:

- Each **Indian city** is represented as a node or vertex.
- Each **road connecting two cities** is represented as an edge.
- The **road distance in kilometres** is used as the weight of the edge.

Dijkstra's algorithm is then used to find the shortest distance from one selected source city to all other reachable cities.

In Artificial Intelligence, this approach is closely related to **Uniform-Cost Search (UCS)** because both expand the state with the lowest accumulated path cost. In theoretical computer science, the corresponding shortest-path algorithm is commonly known as **Dijkstra's algorithm**.

3. Problem Statement

The objective of this project is to implement Dijkstra's algorithm on a weighted graph representing Indian cities and their road distances.

Given:

- A collection of Indian cities.
- Roads connecting the cities.
- Distance associated with each road.

- A source city.

The program should determine:

1. The shortest road distance from the source city to every other city.
 2. The actual shortest route between two selected cities.
 3. The total distance of that route.
 4. A visual representation of the shortest distances.
-

4. Aim

To implement Dijkstra's algorithm to find the shortest road distances between Indian cities using a weighted graph and demonstrate the relationship between Dijkstra's algorithm and Uniform-Cost Search.

5. Objectives

The main objectives of this project are:

- To understand weighted graphs.
 - To represent Indian cities as graph nodes.
 - To represent roads as weighted edges.
 - To obtain road-distance data from an open-source dataset.
 - To construct a graph using Python.
 - To implement Dijkstra's algorithm from scratch.
 - To calculate shortest distances from a source city.
 - To reconstruct the actual shortest path.
 - To display the results in tabular form.
 - To visualize shortest distances using a bar graph.
 - To understand the relationship between Dijkstra's algorithm and Uniform-Cost Search.
-

6. Background

6.1 Graph

A graph is a mathematical structure consisting of:

$$G = (V, E)$$

where:

- V represents the set of vertices.
- E represents the set of edges.

In this project:

$$V = \{\text{Indian cities}\}$$

and

$$E = \{\text{roads connecting cities}\}$$

For example:

```
Delhi ----- Agra
 |             |
 |             |
Jaipur ----- Lucknow
```

Here, Delhi, Agra, Jaipur, and Lucknow are vertices.

The lines connecting them represent roads.

7. Weighted Graph

A normal graph only tells us whether two cities are connected.

A **weighted graph** also tells us the cost associated with travelling between them.

In this project, the cost is the road distance.

For example:

```
Delhi ----- 233 km ----- Agra
```

The edge between Delhi and Agra has a weight of 233 km.

Therefore, the graph can be represented as:

```
Delhi → Agra = 233 km
```

8. Why Dijkstra's Algorithm?

Suppose we want to travel from Delhi to Mumbai.

There could be several possible routes:

Delhi → Jaipur → Ahmedabad → Mumbai

or

Delhi → Agra → Ahmedabad → Mumbai

or another combination of cities.

Simply choosing the city that appears geographically closest does not necessarily give the shortest total route.

Dijkstra's algorithm considers the **total distance travelled so far**.

It repeatedly chooses the city whose currently known path from the source has the smallest total cost.

Therefore, it is appropriate for road-distance problems where all edge weights are non-negative.

9. Dijkstra and Uniform-Cost Search

Dijkstra's algorithm and Uniform-Cost Search are closely related.

In AI terminology, Uniform-Cost Search uses:

$$f(n) = g(n)$$

where:

- $g(n)$ = cost of reaching node n from the starting node.
- $f(n)$ = evaluation value used to decide which node should be expanded.

Dijkstra's algorithm also selects the node having the smallest accumulated distance.

Therefore:

$\text{Uniform-Cost Search} \approx \text{Dijkstra's Algorithm}$
--

for a graph with non-negative edge costs.

The important point is that no heuristic function is used.

For example, A* would use:

$$f(n) = g(n) + h(n)$$

whereas Dijkstra/UCS uses:

$$f(n) = g(n)$$

10. Dataset Used

For the implementation, an open-source dataset containing Indian city road-distance information is used.

The dataset contains information similar to:

Origin	Destination	Distance
Agra	Delhi	233 km
Ahmedabad	Mumbai	524 km
Bengaluru	Hyderabad	570 km
Delhi	Jaipur	268 km

The dataset is used only as the source of the graph's road connections and distances.

Note: The dataset used for the demonstration is a city-pair dataset and should not be described as a complete representation of every road in India. A complete nationwide road network would require a much larger GIS/OpenStreetMap-based dataset.

11. Tools and Technologies

The project was implemented using:

Programming Language

Python

Platform

Google Colab

Libraries

- Pandas
- Heapq
- Matplotlib
- Requests
- StringIO

Purpose of each library

Library	Purpose
Pandas	Loading and processing dataset
Heapq	Implementing priority queue
Matplotlib	Creating graphs
Requests	Downloading dataset
StringIO	Reading downloaded CSV data

12. Step 1 — Importing Required Libraries

The first step is to import the Python libraries required for the project.

```
import pandas as pd
import heapq
import matplotlib.pyplot as plt
import requests
from io import StringIO
```

Explanation

`pandas` is used to read and manipulate the dataset.

`heapq` provides a priority queue implementation. This is important because Dijkstra's algorithm needs to select the city with the smallest current distance.

`matplotlib` is used to visualize the final results.

`requests` is used to download the open-source dataset automatically.

`StringIO` allows the downloaded CSV text to be treated like a file.

13. Step 2 — Downloading the Dataset

Instead of manually uploading a CSV file, the dataset can be downloaded directly in Google Colab.

```
url = "https://gist.githubusercontent.com/kshipra-
jadav/4697f52b61dc246e37441ca88c59cfa5/raw/indian-cities-dataset.csv"

response = requests.get(url)

if response.status_code == 200:
    df = pd.read_csv(StringIO(response.text))
    print("Dataset downloaded successfully!")
else:
```

```
print("Download failed.")
```

Explanation

The program sends a request to the dataset URL.

If the server returns status code 200, the file was successfully obtained.

The CSV data is then loaded into a Pandas DataFrame.

This approach avoids the earlier:

```
FileNotFoundError
```

because the program does not depend on a file already existing at:

```
/content/indian-cities-dataset.csv
```

14. Step 3 — Examining the Dataset

After loading the data, we inspect its structure.

```
print("Dataset shape:", df.shape)

print("Columns:")
print(df.columns.tolist())

display(df.head(10))
```

This allows us to understand:

- Number of records.
- Available columns.
- Data format.
- Whether the dataset was loaded correctly.

The important columns are:

```
Origin
Destination
Distance
```

15. Step 4 — Cleaning the Dataset

Before constructing the graph, the data needs to be cleaned.

```
df = df.rename(columns={
    "Origin": "source",
```

```
        "Destination": "target",
        "Distance": "distance"
    })

df = df.dropna()

df["distance"] = pd.to_numeric(
    df["distance"],
    errors="coerce"
)

df = df.dropna()
```

Why is cleaning necessary?

Real-world datasets can contain:

- Missing values.
- Incorrect data types.
- Empty records.
- Text values where numerical values are expected.

Dijkstra's algorithm needs the road distance to be numerical, so the distance column is converted into a numeric format.

16. Step 5 — Creating the Graph

The cleaned dataset is converted into a graph.

```
graph = {}

for _, row in df.iterrows():

    source = row["source"]
    target = row["target"]
    distance = float(row["distance"])

    if source not in graph:
        graph[source] = []

    if target not in graph:
        graph[target] = []

    graph[source].append((target, distance))
    graph[target].append((source, distance))
```

17. Why Are Roads Added Twice?

Notice that the program contains:

```
graph[source].append((target, distance))
graph[target].append((source, distance))
```

This means the road is treated as **bidirectional**.

For example:

Delhi ----- 233 km ----- Agra

The graph stores:

```
Delhi → Agra = 233 km
Agra → Delhi = 233 km
```

This is appropriate for a simplified road-distance graph where travelling in either direction is allowed.

In a real navigation system, some roads may be one-way, so the graph would need to be directed.

18. Graph Representation

Internally, the graph looks approximately like:

```
Delhi:
  Agra      → 233 km
  Jaipur    → 268 km
  Lucknow   → 555 km
```

```
Mumbai:
  Ahmedabad → 524 km
  Pune      → 150 km
```

```
Bengaluru:
  Hyderabad → 570 km
  Chennai   → 350 km
```

This is called an **adjacency-list representation**.

It is efficient because each city stores its connected neighbours and corresponding road distances.

19. Step 6 — Implementing Dijkstra's Algorithm

The main algorithm is:

```

def dijkstra(graph, source):

    distances = {
        city: float("inf")
        for city in graph
    }

    previous = {
        city: None
        for city in graph
    }

    distances[source] = 0

    priority_queue = [(0, source)]

    while priority_queue:

        current_distance, current_city = heapq.heappop(
            priority_queue
        )

        if current_distance > distances[current_city]:
            continue

        for neighbour, road_distance in graph[current_city]:

            new_distance = (
                current_distance + road_distance
            )

            if new_distance < distances[neighbour]:

                distances[neighbour] = new_distance

                previous[neighbour] = current_city

                heapq.heappush(
                    priority_queue,
                    (new_distance, neighbour)
                )

    return distances, previous

```

20. Understanding the Algorithm Step-by-Step

The algorithm may look complicated initially, but its basic idea is simple.

Step 1: Give every city infinite distance

Initially, we don't know how far the cities are.

Therefore:

```
Delhi = 0
Agra = ∞
Mumbai = ∞
Hyderabad = ∞
Chennai = ∞
```

The source city gets:

```
Delhi = 0
```

because the distance from Delhi to itself is zero.

Step 2: Put the source into the priority queue

```
priority_queue = [(0, source)]
```

If the source is Delhi:

```
(0, Delhi)
```

Step 3: Select the smallest distance

The priority queue always gives us the city with the smallest known distance.

Initially:

```
Delhi = 0
```

so Delhi is selected.

Step 4: Check its neighbours

Suppose Delhi is connected to:

```
Agra = 233 km
Jaipur = 268 km
Lucknow = 555 km
```

The algorithm calculates:

```
Delhi → Agra = 0 + 233 = 233
```

```
Delhi → Jaipur = 0 + 268 = 268
```

```
Delhi → Lucknow = 0 + 555 = 555
```

The distances are updated.

21. Relaxation

One of the most important concepts in Dijkstra's algorithm is **relaxation**.

Suppose we currently have:

Delhi → Mumbai = 1400 km

Later, we discover another route:

Delhi → Jaipur → Ahmedabad → Mumbai

and its total distance is:

1350 km

Since:

$$1350 < 1400$$

we replace:

1400 km

with:

1350 km

Mathematically:

$$d(v) = \min(d, (v)d(u) + w(u, v))$$

This is exactly what the Python code does:

```
if new_distance < distances[neighbour]:  
    distances[neighbour] = new_distance
```

22. Priority Queue

The priority queue is an important part of the implementation.

It stores:

(distance, city)

For example:

```
(233, Agra)
(268, Jaipur)
(555, Lucknow)
```

The smallest value is processed first.

This means Dijkstra always focuses on the city with the currently smallest accumulated cost.

23. Step 7 — Finding Shortest Distances from Delhi

The program can be executed using:

```
source = "Delhi"

distances, previous = dijkstra(
    graph,
    source
)
```

The result gives the shortest known distance from Delhi to every reachable city.

For example:

```
Delhi      : 0 km
Agra       : ...
Jaipur     : ...
Lucknow    : ...
Mumbai     : ...
Hyderabad  : ...
Bengaluru  : ...
Chennai    : ...
```

The exact values depend on the connections contained in the selected dataset.

24. Step 8 — Finding the Actual Route

Finding the distance alone is not always enough.

For example, if the program tells us:

```
Delhi → Mumbai = 1350 km
```

we may also want to know **how we get there**.

For this purpose, the algorithm stores the previous city.

```
previous[neighbour] = current_city
```

This creates a chain of cities.

25. Path Reconstruction

The following function reconstructs the route:

```
def get_path(previous, source, destination):  
    path = []  
    current = destination  
    while current is not None:  
        path.append(current)  
        current = previous[current]  
    path.reverse()  
    if path[0] != source:  
        return []  
    return path
```

Suppose the previous-city information is:

```
Mumbai ← Ahmedabad  
Ahmedabad ← Jaipur  
Jaipur ← Delhi
```

The program reconstructs:

```
Mumbai  
← Ahmedabad  
← Jaipur  
← Delhi
```

Then reverses it:

```
Delhi → Jaipur → Ahmedabad → Mumbai
```

26. Step 9 — Finding a Specific Route

For example:

```
source = "Delhi"  
destination = "Mumbai"
```

```

distances, previous = dijkstra(
    graph,
    source
)

path = get_path(
    previous,
    source,
    destination
)

```

The result can then be displayed as:

Shortest Path:

Delhi → Jaipur → Ahmedabad → Mumbai

Shortest Road Distance:

XXXX km

Again, the actual route and distance are determined by the dataset.

27. Step 10 — Creating a Result Table

Instead of displaying the results one by one, we can create a DataFrame.

```

results = []

for city in sorted(distances):

    path = get_path(
        previous,
        source,
        city
    )

    results.append({
        "Source": source,
        "Destination": city,
        "Shortest Distance (km)": distances[city],
        "Shortest Path": " → ".join(path)
    })

results_df = pd.DataFrame(results)

display(results_df)

```

The table contains:

Source	Destination	Shortest Distance	Shortest Path
Delhi	Agra	...	Delhi → Agra

Source	Destination	Shortest Distance	Shortest Path
Delhi	Jaipur		... Delhi → Jaipur
Delhi	Mumbai		... Delhi → ... → Mumbai
Delhi	Hyderabad		... Delhi → ... → Hyderabad

This makes the results easier to understand and present.

28. Step 11 — Visualization

A bar chart can be used to compare the shortest distances.

```
plot_df = results_df[
    results_df["Destination"] != source
].sort_values(
    "Shortest Distance (km)"
)

plt.figure(figsize=(14, 6))

plt.bar(
    plot_df["Destination"],
    plot_df["Shortest Distance (km)"]
)

plt.xlabel("Destination City")
plt.ylabel("Shortest Road Distance (km)")

plt.title(
    "Dijkstra's Shortest Road Distances from "
    + source
)

plt.xticks(rotation=90)

plt.tight_layout()
plt.show()
```

29. Interpretation of the Graph

The bar graph shows the shortest calculated road distance from the selected source city to each destination city.

A shorter bar indicates a smaller shortest-path distance.

A taller bar indicates a larger shortest-path distance.

The graph provides a simple visual way to compare the distances calculated by Dijkstra's algorithm.

30. Interactive Implementation

The program can also allow the user to enter the source city.

```
print("Available cities:")
print(", ".join(sorted(graph.keys()))))

source = input(
    "\nEnter source city: "
).strip()

if source not in graph:
    print("City not found!")
else:
    distances, previous = dijkstra(
        graph,
        source
    )

    print(
        "\nShortest distances from",
        source
    )

    for city, distance in sorted(
        distances.items(),
        key=lambda x: x[1]
    ):
        print(
            f"{city:20} : {distance:.0f} km"
        )
```

This makes the project more user-friendly because the source city does not have to be fixed to Delhi.

31. Complete Working Flow

The complete process can be summarized as:

```
Open-source Dataset
  ↓
Download Dataset
  ↓
Load using Pandas
  ↓
Clean Dataset
  ↓
```

```
Extract Source, Destination and Distance
↓
Create Weighted Graph
↓
Select Source City
↓
Initialize Distances
↓
Create Priority Queue
↓
Select Minimum-Cost City
↓
Check Neighbours
↓
Relax Edges
↓
Update Shortest Distances
↓
Repeat Until Queue is Empty
↓
Shortest Distance to All Cities
↓
Reconstruct Shortest Paths
↓
Display Table
↓
Generate Visualization
```

32. Pseudocode

DIJKSTRA(Graph, Source)

```
For every city:
    distance[city] = infinity
    previous[city] = NULL
```

```
distance[Source] = 0
```

```
Insert Source into priority queue
```

```
While priority queue is not empty:
```

```
    Remove city with minimum distance
```

```
    For every neighbour of current city:
```

```
        new_distance =
            current_distance + road_distance
```

```
        If new_distance < current distance:
```

```
            Update distance
```

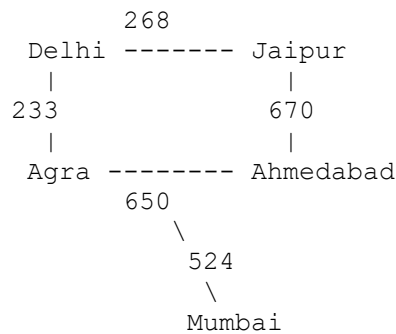
```
            Store previous city
```

```
            Insert neighbour into priority queue
```

```
Return distances and previous
```

33. Example of the Working Process

Consider a simplified graph:



Suppose Delhi is the source.

Initially:

```
Delhi = 0
Agra = ∞
Jaipur = ∞
Ahmedabad = ∞
Mumbai = ∞
```

After processing Delhi:

```
Agra = 233
Jaipur = 268
```

The algorithm selects Agra because:

$$233 < 268$$

It then checks Agra's neighbours.

The process continues until all reachable cities have their minimum distances.

The important point is that the algorithm does not simply choose the road with the smallest individual distance. It considers the **total path cost**.

34. Advantages

1. Simple and systematic

Dijkstra follows a clear procedure for finding minimum-cost paths.

2. Suitable for road networks

Road distances are normally non-negative, which satisfies Dijkstra's requirement.

3. Finds paths to multiple destinations

A single execution from a source can determine shortest distances to all reachable cities.

4. Provides actual routes

By storing the previous city, the algorithm can reconstruct the complete route.

5. Efficient with a priority queue

The priority queue avoids repeatedly searching through all cities to find the smallest distance.

35. Limitations

1. Does not use geographical heuristics

Dijkstra does not know whether a city is geographically close to the destination.

2. Can explore many unnecessary cities

For a very large road network, it may process many nodes that are not necessary for a single source-to-destination query.

3. Requires non-negative edge weights

Dijkstra's algorithm is not suitable for graphs containing negative edge weights.

4. Dataset limitation

The demonstration dataset is not a complete representation of India's entire road network. A real navigation system would require much more detailed road-network data.

5. Real-world road conditions are not considered

The basic implementation considers only distance. It does not consider:

- Traffic
- Road quality
- Toll charges
- Weather
- Construction
- Accidents

- Travel time
- One-way restrictions

Therefore, the shortest distance calculated by the program is not necessarily the fastest real-world route.

36. Time Complexity

When a priority queue is used, the approximate time complexity is:

$$O((V + E) \log V)$$

where:

- V = number of vertices/cities.
- E = number of edges/roads.

The space complexity is approximately:

$$O(V + E)$$

because the graph, distances, previous-city information, and priority queue need to be stored.

37. Dijkstra vs Uniform-Cost Search

Feature	Dijkstra	Uniform-Cost Search
Main area	Graph algorithms	Artificial Intelligence
Purpose	Shortest path	Lowest-cost path
Cost considered	Total path distance	Total path cost
Heuristic	No	No
Priority	Smallest distance	Smallest path cost
Edge costs	Non-negative	Non-negative
Data structure	Priority queue	Priority queue
Evaluation	$g(n)$	$g(n)$

Thus, the algorithm implemented in this project can be understood as **Dijkstra's shortest-path algorithm or Uniform-Cost Search applied to a road-distance state space.**

38. Real-World Applications

Dijkstra's algorithm has many applications.

Navigation

Finding shortest routes between cities.

GPS systems

Calculating routes through road networks.

Network routing

Finding minimum-cost routes between computers or routers.

Transportation

Finding efficient routes for buses, trains, and delivery vehicles.

Robotics

Finding paths through environments represented as graphs.

Game development

Finding paths for characters through maps.

Logistics

Finding routes that minimize transportation distance or cost.

39. Expected Result

After running the program, the user can select a source city such as Delhi.

The program then:

1. Reads the city-road dataset.
2. Constructs the weighted graph.
3. Sets Delhi's distance to zero.
4. Uses a priority queue.
5. Expands the city with the smallest accumulated distance.
6. Performs edge relaxation.
7. Calculates shortest distances to all reachable cities.

8. Stores the previous city for each shortest path.
 9. Reconstructs routes when required.
 10. Displays the results in a table.
 11. Generates a bar chart of shortest distances.
-

40. Observation

The implementation demonstrates that Dijkstra's algorithm can efficiently determine minimum-cost road paths in a weighted graph. The algorithm continuously selects the city with the smallest accumulated distance and updates neighbouring cities through relaxation. By storing the previous city, the complete shortest route can also be reconstructed. The experiment also demonstrates the close relationship between Dijkstra's algorithm and Uniform-Cost Search, since both prioritize states according to their accumulated path cost.

41. Conclusion

This project successfully implements Dijkstra's algorithm for an Indian-city road network.

The cities are represented as vertices and road connections are represented as weighted edges, with road distance used as the edge weight. An open-source dataset is loaded and converted into an adjacency-list graph.

A priority queue is then used to implement Dijkstra's algorithm efficiently. Starting from a selected source city, the algorithm calculates the minimum known distance to each reachable city and updates these distances whenever a shorter route is discovered.

The project also reconstructs the actual shortest route between cities and presents the results in both tabular and graphical form.

Overall, the implementation provides a practical demonstration of how **graph search algorithms can be applied to real-world route-finding problems**. It also shows why Dijkstra's algorithm is closely related to **Uniform-Cost Search in Artificial Intelligence**, where the next state is selected based on the lowest accumulated path cost.
